SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1110

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing Techniques (BL4, BL5)
Assessment Tool Weightage: 25%

QUESTIONS: 05 **Total Marks:25**

Annexure A

Error Analysis Task

Code of Conduct:

I Sanjay J / 192511331 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Sanjay J

RUBRICS FOR CALCULATION:

Error Analysis Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Annexure A

Error Analysis Task

Q1. Scenario: Online Banking System

Modules:
User Login, Fund Transfer, Account Verification

Question:

Write pseudocode for a quality assurance testing process that validates secure user authentication and fund transfer operations. Include input validation, transaction verification, exception handling, and test result logging.

Q2. Scenario: E-Commerce Shopping Platform

Features:
Product Search, Shopping Cart, Payment Gateway

Question:

Design pseudocode for a usability testing evaluation that measures customer interaction efficiency, including product search time, checkout completion time, and navigation complexity. Explain how the algorithm collects and analyzes user behavior data.

Q3. Scenario: Hospital Management System

Modules:
Patient Registration, Appointment Scheduling, Medical Records

Question:

Write pseudocode to generate and execute test cases for appointment scheduling functionality. Include validation of doctor availability, appointment conflicts, patient information accuracy, and system responses to invalid requests.

Q4. Scenario: Airline Reservation System

Modules:
Passenger Registration, Flight Booking, Seat Allocation

Question:

Develop pseudocode for a constraint-based testing approach where booking requests must satisfy conditions such as valid passenger information, seat availability, and payment confirmation. Show how invalid booking scenarios are detected and handled.

Q5. Scenario: Smart Library Management System

Features:
Book Search, Book Reservation, Borrow/Return Services

Question:

Write pseudocode for integrating quality assurance and usability testing where the system evaluates both functional correctness (reservation and borrowing operations) and user experience factors such as response time, ease of navigation, and accessibility.

Q1. Online Banking System

Modules / Features: User Login, Fund Transfer, Account Verification

Write pseudocode for a quality assurance testing process that validates secure user authentication and fund transfer operations. Include input validation, transaction verification, exception handling, and test result logging.

1. Pseudocode

```
BEGIN QA_Test_OnlineBanking

 // ---- MODULE 1: User Login / Authentication ----
 FUNCTION testUserLogin(username, password):
    TRY
       IF username IS EMPTY OR password IS EMPTY THEN
          RAISE InputValidationException("Credentials cannot be empty")
       END IF
       IF NOT matchesPattern(username, VALID_USERNAME_REGEX) THEN
          RAISE InputValidationException("Invalid username format")
       END IF

       storedHash = DB.getPasswordHash(username)
       IF storedHash IS NULL THEN
          RAISE AuthenticationException("User not found")
       END IF

       IF hash(password) != storedHash THEN
          attemptCount = incrementFailedAttempts(username)
          IF attemptCount >= MAX_ATTEMPTS THEN
             lockAccount(username)
             RAISE AccountLockedException("Account locked after repeated failures")
          END IF
          RAISE AuthenticationException("Invalid credentials")
       END IF

       sessionToken = generateSecureToken(username)
       logTestResult("Login", username, "PASS", currentTimestamp())
       RETURN sessionToken

    CATCH InputValidationException AS e
       logTestResult("Login", username, "FAIL - " + e.message, currentTimestamp())
       RETURN NULL
    CATCH AuthenticationException AS e
       logTestResult("Login", username, "FAIL - " + e.message, currentTimestamp())
       RETURN NULL
    CATCH AccountLockedException AS e
       logTestResult("Login", username, "FAIL - " + e.message, currentTimestamp())
       RETURN NULL
```

```
    END TRY
  END FUNCTION

  // ---- MODULE 2: Account Verification ----
  FUNCTION testAccountVerification(sessionToken, accountId):
    TRY
      IF NOT isValidSession(sessionToken) THEN
        RAISE SessionException("Invalid or expired session")
      END IF
      account = DB.getAccount(accountId)
      IF account IS NULL OR account.status != "ACTIVE" THEN
        RAISE AccountVerificationException("Account inactive or not found")
      END IF
      logTestResult("AccountVerification", accountId, "PASS", currentTimestamp())
      RETURN account
    CATCH SessionException, AccountVerificationException AS e
      logTestResult("AccountVerification", accountId, "FAIL - " + e.message,
currentTimestamp())
      RETURN NULL
    END TRY
  END FUNCTION

  // ---- MODULE 3: Fund Transfer ----
  FUNCTION testFundTransfer(sessionToken, fromAcc, toAcc, amount):
    TRY
      account = testAccountVerification(sessionToken, fromAcc)
      IF account IS NULL THEN
        RAISE TransferException("Source account verification failed")
      END IF
      IF amount <= 0 THEN
        RAISE InputValidationException("Transfer amount must be positive")
      END IF
      IF account.balance < amount THEN
        RAISE InsufficientFundsException("Balance too low for transfer")
      END IF
      IF DB.getAccount(toAcc) IS NULL THEN
        RAISE InputValidationException("Destination account does not exist")
      END IF
```

```
BEGIN_TRANSACTION
            debit(fromAcc, amount)
            credit(toAcc, amount)
        COMMIT_TRANSACTION

        logTestResult("FundTransfer", fromAcc + "->" + toAcc, "PASS",
currentTimestamp())
        RETURN "SUCCESS"

    CATCH InsufficientFundsException, InputValidationException, TransferException AS
e
        ROLLBACK_TRANSACTION
        logTestResult("FundTransfer", fromAcc + "->" + toAcc, "FAIL - " + e.message,
currentTimestamp())
        RETURN "FAILURE"
    END TRY
 END FUNCTION

 // ---- Driver: run test suite and summarize ----
 FUNCTION runTestSuite(testCases):
    FOR EACH tc IN testCases DO
        result = executeTestCase(tc)
        appendToTestReport(result)
    END FOR
    generateSummaryReport()
 END FUNCTION

END QA_Test_OnlineBanking
```

2. Process Flow Diagram

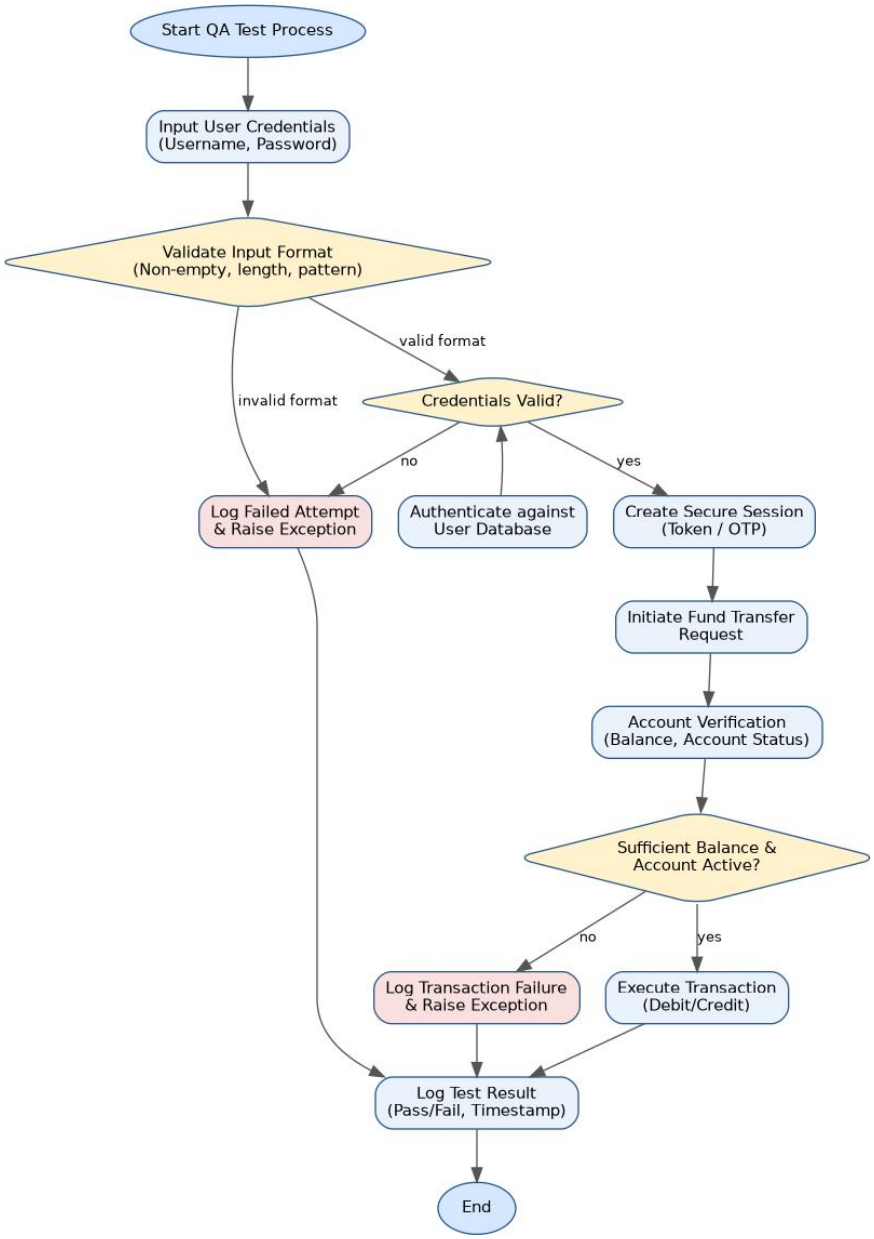

Figure 1: Process flow for Online Banking System testing procedure

3. Error Identification, Root Cause Analysis & Correction

Error Identified	Root Cause Analysis	Correction & Justification
Missing boundary check on transfer amount (e.g., negative or zero values accepted)	Developer validated only for NULL/empty input and overlooked numeric range constraints during unit design	Add an explicit range check (amount > 0 and amount <= dailyLimit) before the transaction block; unit-test with boundary values (0, -1, MAX_LIMIT+1)
Session token not re-validated before the fund-transfer step, allowing a stale/expired session to initiate a transfer	Session validation was implemented only in the login module and assumed to persist without re-checking at each sensitive operation	Re-verify session validity at the start of every privileged function (defense-in-depth); justified because financial operations must never trust a cached authentication state
No rollback on partial transaction failure (debit succeeds but credit fails)	Debit and credit operations were coded as two independent steps instead of an atomic unit	Wrap debit/credit inside BEGIN_TRANSACTION / COMMIT_TRANSACTION with ROLLBACK_TRANSACTION on any exception, guaranteeing ACID consistency
Failed login attempts not rate-limited, exposing the system to brute-force attacks	Root cause is absence of an attempt-counter and lockout policy tied to authentication logic	Introduce MAX_ATTEMPTS with account lockout and exponential back-off, justified by OWASP authentication best practices

4. Application of OOAD / QA Concepts

- Encapsulation: authentication, verification, and transfer are separated into independent, cohesive functions with their own responsibility.
- Exception handling hierarchy models real-world banking failure modes (InputValidationException, AuthenticationException, InsufficientFundsException) instead of one generic error.
- Transaction integrity (ACID) is applied to the fund-transfer use case to prevent partial updates.
- Test logging supports traceability and audit requirements typical of quality assurance in regulated financial systems.

Q2. E-Commerce Shopping Platform

Modules / Features: Product Search, Shopping Cart, Payment Gateway

Design pseudocode for a usability testing evaluation that measures customer interaction efficiency, including product search time, checkout completion time, and navigation complexity. Explain how the algorithm collects and analyzes user behaviour data.

1. Pseudocode

```
BEGIN Usability_Test_ECommerce

 STRUCTURE SessionMetrics:
    userId, searchTime, checkoutTime, clickCount,
    backNavigationCount, errorCount, deviceType
 END STRUCTURE

 FUNCTION startSession(userId):
    session = new SessionMetrics(userId)
    session.startTime = currentTimestamp()
    logEvent("SESSION_START",  userId)
    RETURN session
 END FUNCTION

 // ---- Measure Product Search Efficiency ----
 FUNCTION measureSearch(session, query):
    t1 = currentTimestamp()
    results = ProductSearch.execute(query)
    t2 = currentTimestamp()
    session.searchTime = t2 - t1
    session.searchRelevance = computeRelevanceScore(results, query)
    IF results IS EMPTY THEN
       session.errorCount += 1
       logEvent("SEARCH_ZERO_RESULTS", query)
    END IF
    RETURN results
 END FUNCTION

 // ---- Track Navigation Complexity ----
 FUNCTION trackNavigation(session, pageEvent):
    session.clickCount += 1
    IF pageEvent.type == "BACK" THEN
       session.backNavigationCount += 1
    END IF
    logEvent("NAV", pageEvent)
 END FUNCTION

 // ---- Measure Checkout Completion Time ----
```

```
FUNCTION measureCheckout(session, cart):
    t1 = currentTimestamp()
    IF cart IS EMPTY THEN
        session.errorCount += 1
        RETURN "ABANDONED_EMPTY_CART"
    END IF
    status = PaymentGateway.process(cart)
    t2 = currentTimestamp()
    session.checkoutTime = t2 - t1
    IF status != "SUCCESS" THEN
        session.errorCount += 1
    END IF
    RETURN status
END FUNCTION

// ---- Compute Navigation Complexity Score ----
FUNCTION computeComplexityScore(session):
    // Weighted formula combining clicks, back-tracks and errors
    score = (session.clickCount * 0.4) +
            (session.backNavigationCount * 1.5) +
            (session.errorCount * 2.0)
    RETURN score
END FUNCTION

// ---- Analyze Collected Behaviour Data ----
FUNCTION analyzeSession(session):
    complexity = computeComplexityScore(session)
    report = {
        searchTime: session.searchTime,
        checkoutTime: session.checkoutTime,
        complexityScore: complexity,
        errorCount: session.errorCount
    }
    IF session.searchTime > SEARCH_TIME_THRESHOLD OR
       session.checkoutTime > CHECKOUT_TIME_THRESHOLD OR
       complexity > COMPLEXITY_THRESHOLD THEN
       report.verdict = "USABILITY_ISSUE"
       suggestUXImprovement(report)
    ELSE
        report.verdict = "PASS"
    END IF
    persistToAnalyticsStore(report)
    RETURN report
END FUNCTION

// ---- Aggregate across many users for trend analysis ----
```

```
 FUNCTION aggregateUsabilityTrends(allSessions):
    avgSearchTime = average(allSessions.searchTime)
    avgCheckoutTime = average(allSessions.checkoutTime)
    avgComplexity = average(allSessions.complexityScore)
    RETURN buildTrendReport(avgSearchTime, avgCheckoutTime, avgComplexity)
 END FUNCTION

END Usability_Test_ECommerce
```

2. Process Flow Diagram

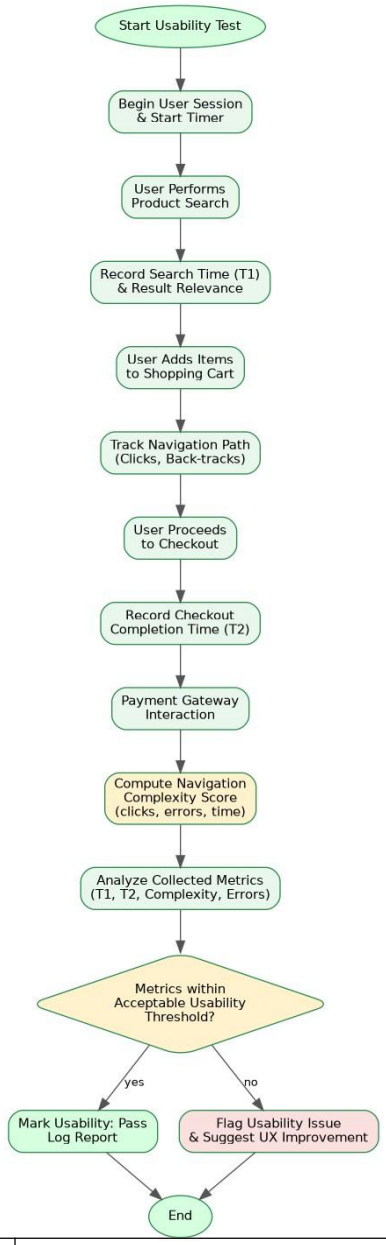

Figure 2: Process flow for E-Commerce Shopping Platform testing procedure

3. Error Identification, Root Cause Analysis & Correction

Error Identified	Root Cause Analysis	Correction & Justification
Complexity score formula does not normalize for	The weighting scheme (clicks, back-navigation,	Normalize the score against expected task benchmarks

Error Identified	Root Cause Analysis	Correction & Justification
session length, unfairly penalizing users who legitimately browse longer	errors) was designed without accounting for task type or intended session duration	(e.g., complexity per minute) and justify weights using prior UX research or A/B baseline data
Zero-result searches are logged but not distinguished from slow-but-successful searches when computing the verdict	The analyzeSession function only checks time thresholds and complexity, not search-quality failures	Add a dedicated relevance/failure flag to the verdict logic so zero-result and low-relevance searches are separately reported, giving more actionable UX feedback
Abandoned/empty-cart checkouts are counted as an error but not separated from payment-gateway failures in analytics	Root cause is a single generic errorCount counter used for multiple distinct failure types	Split errorCount into categorized counters (cartAbandonment, paymentFailure, searchFailure) — justified because each requires a different UX fix

4. Application of OOAD / QA Concepts

- Behavioural instrumentation pattern: every user action (search, click, checkout) is timestamped and logged for later analysis, a core usability-testing technique.
- Quantitative UX metrics (search time, checkout time, navigation complexity) are converted into an objective composite score rather than relying on subjective opinion.
- Threshold-based verdicts operationalize usability heuristics (efficiency, learnability) into automatable pass/fail criteria.
- Aggregation across sessions supports trend analysis, which is essential for continuous usability improvement in an e-commerce platform.

Q3. Hospital Management System

Modules / Features: Patient Registration, Appointment Scheduling, Medical Records

Write pseudocode to generate and execute test cases for appointment scheduling functionality. Include validation of doctor availability, appointment conflicts, patient information accuracy, and system responses to invalid requests.

1. Pseudocode

```
BEGIN TestSuite_AppointmentScheduling

 // ---- Test Case Generation ----
 FUNCTION generateTestCases():
    testCases = []
    testCases.ADD(TC("TC01", validPatient, validDoctor, futureSlot, EXPECT="CONFIRMED"))
    testCases.ADD(TC("TC02", missingPatientId, validDoctor, futureSlot, EXPECT="REJECTED"))
    testCases.ADD(TC("TC03", validPatient, validDoctor, pastDateSlot, EXPECT="REJECTED"))
    testCases.ADD(TC("TC04", validPatient, unavailableDoctor, busySlot, EXPECT="REJECTED"))
    testCases.ADD(TC("TC05", validPatient, validDoctor, doubleBookedSlot, EXPECT="CONFLICT"))
    testCases.ADD(TC("TC06", invalidFormatPatientId, validDoctor, futureSlot, EXPECT="REJECTED"))
    RETURN testCases
 END FUNCTION

 // ---- Patient Information Accuracy Check ----
 FUNCTION validatePatientInfo(patient):
    IF patient.id IS NULL OR NOT matchesPattern(patient.id, ID_REGEX) THEN
       RETURN FAIL("Invalid or missing patient ID")
    END IF
    IF patient.name IS EMPTY OR patient.dob IS INVALID_DATE THEN
       RETURN FAIL("Incomplete demographic data")
    END IF
    RETURN PASS()
 END FUNCTION

 // ---- Doctor Availability Check ----
 FUNCTION checkDoctorAvailability(doctorId, slot):
    schedule = DB.getDoctorSchedule(doctorId)
    IF schedule IS NULL THEN
       RETURN FAIL("Doctor not found")
    END IF
    IF NOT schedule.isWorkingAt(slot) THEN
```

```
        RETURN FAIL("Doctor unavailable at requested slot")
    END IF
    RETURN PASS()
END FUNCTION

// ---- Appointment Conflict Detection ----
FUNCTION checkConflict(doctorId, slot):
    existingAppointments = DB.getAppointments(doctorId, slot.date)
    FOR EACH appt IN existingAppointments DO
        IF appt.overlaps(slot) THEN
            RETURN CONFLICT("Slot already booked: " + appt.id)
        END IF
    END FOR
    RETURN PASS()
END FUNCTION

// ---- Execute a single test case ----
FUNCTION executeTestCase(tc):
    TRY
        patientResult = validatePatientInfo(tc.patient)
        IF patientResult.status == "FAIL" THEN
            RAISE ValidationException(patientResult.message)
        END IF

        availResult = checkDoctorAvailability(tc.doctorId, tc.slot)
        IF availResult.status == "FAIL" THEN
            RAISE AvailabilityException(availResult.message)
        END IF

        conflictResult = checkConflict(tc.doctorId, tc.slot)
        IF conflictResult.status == "CONFLICT" THEN
            RAISE ConflictException(conflictResult.message)
        END IF

        bookAppointment(tc.patient, tc.doctorId, tc.slot)
        actual = "CONFIRMED"

    CATCH ValidationException, AvailabilityException AS e
        actual = "REJECTED"
    CATCH ConflictException AS e
        actual = "CONFLICT"
    END TRY

    passed = (actual == tc.expected)
    logTestResult(tc.id, tc.expected, actual, passed)
    RETURN passed
```

```
END FUNCTION

// ---- Driver ----
FUNCTION runAllTests():
    testCases = generateTestCases()
    FOR EACH tc IN testCases DO
        executeTestCase(tc)
    END FOR
    generateSummaryReport()
END FUNCTION

END TestSuite_AppointmentScheduling
```

2. Process Flow Diagram

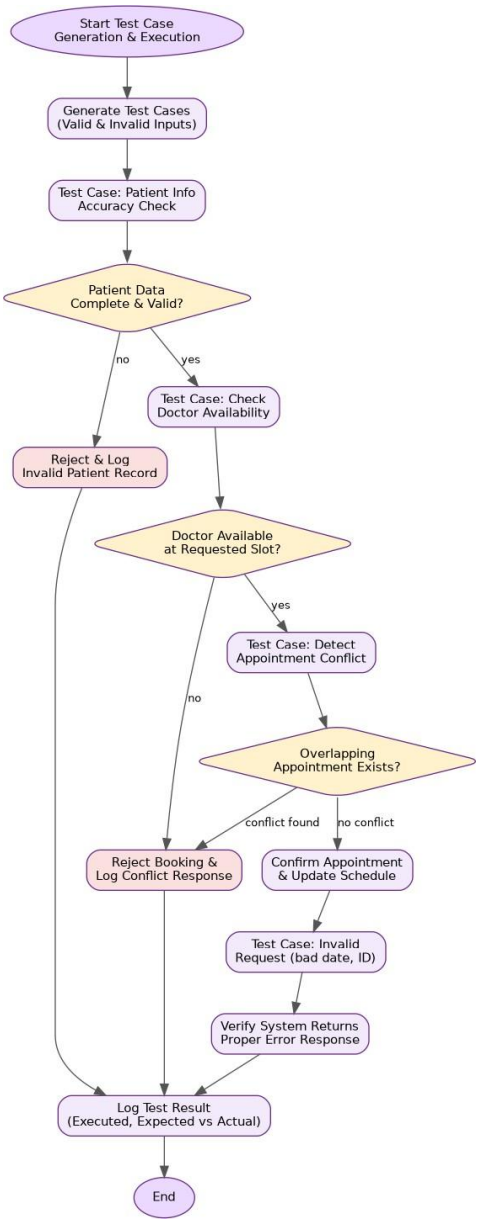

Figure 3: Process flow for Hospital Management System testing procedure

3. Error Identification, Root Cause Analysis & Correction

Error Identified	Root Cause Analysis	Correction & Justification
Past-date appointment slots are not explicitly rejected before reaching the availability check	Doctor-availability logic assumes only future dates are ever passed in, so no separate date-sanity check exists	Add an explicit guard (slot.date >= today) at the start of executeTestCase and raise a distinct ValidationException; justified because it fails fast and gives a clearer error message than an availability mismatch
Conflict detection only checks the requested doctor's schedule, not whether the same patient already has an overlapping appointment with a different doctor	checkConflict was scoped narrowly to doctor-side double-booking during design, overlooking patient-side conflicts	Add a parallel checkPatientConflict(patientId, slot) call so both sides of the booking are validated, improving correctness of the scheduling module
Invalid patient-ID format test case (TC06) and missing-ID test case (TC02) both map to the same generic REJECTED outcome, hiding which validation rule actually failed	validatePatientInfo returns a single FAIL status without an error code distinguishing the specific rule violated	Introduce distinct error codes/messages per validation rule so test logs and QA reports can pinpoint exact root causes instead of a generic rejection

4. Application of OOAD / QA Concepts

- Test-case design uses equivalence partitioning and boundary-value analysis (valid slot, past date, double-booked slot, invalid ID) to maximize defect coverage with a minimal test set.
- Layered validation (patient info -> doctor availability -> conflict check) mirrors good object-oriented separation of concerns across independent modules.
- Exception-driven control flow maps each failure category to a distinct, traceable exception type instead of a single catch-all error.
- Expected-vs-actual comparison in executeTestCase implements the core QA principle of automated pass/fail verification.

Q4. Airline Reservation System

Modules / Features: Passenger Registration, Flight Booking, Seat Allocation

Develop pseudocode for a constraint-based testing approach where booking requests must satisfy conditions such as valid passenger information, seat availability, and payment confirmation. Show how invalid booking scenarios are detected and handled.

1. Pseudocode

```
BEGIN Constraint_Based_Test_AirlineBooking

 STRUCTURE BookingRequest:
    passenger, flightId, seatPreference, paymentInfo
 END STRUCTURE

 // ---- Constraint Definitions ----
 CONSTRAINTS = [
    C1: "Passenger info must be complete and valid",
    C2: "Requested seat must be available on the flight",
    C3: "Payment must be confirmed before seat lock"
 ]

 FUNCTION validatePassenger(passenger):
    IF passenger.name IS EMPTY OR passenger.passportId IS INVALID THEN
       RETURN FAIL("C1 violated: invalid passenger information")
    END IF
    IF passenger.age < 0 OR passenger.age > 120 THEN
       RETURN FAIL("C1 violated: implausible age value")
    END IF
    RETURN PASS()
 END FUNCTION

 FUNCTION checkSeatAvailability(flightId, seatPref):
    flight = DB.getFlight(flightId)
    IF flight IS NULL THEN
       RETURN FAIL("C2 violated: flight does not exist")
    END IF
    availableSeats = flight.getAvailableSeats(seatPref.class)
    IF availableSeats.count == 0 THEN
       RETURN FAIL("C2 violated: no seats available in requested class")
    END IF
    RETURN PASS(availableSeats.first())
 END FUNCTION

 FUNCTION confirmPayment(paymentInfo, amount):
    IF paymentInfo IS NULL OR paymentInfo.cardValid == FALSE THEN
       RETURN FAIL("C3 violated: invalid payment details")
```

```
    END IF
    status = PaymentGateway.charge(paymentInfo, amount)
    IF status != "SUCCESS" THEN
       RETURN FAIL("C3 violated: payment not confirmed")
    END IF
    RETURN PASS()
 END FUNCTION

 // ---- Constraint-Based Booking Evaluation ----
 FUNCTION processBooking(request):
    TRY
       r1 = validatePassenger(request.passenger)
       IF r1.status == "FAIL" THEN RAISE ConstraintException(r1.message) END IF

       r2 = checkSeatAvailability(request.flightId, request.seatPreference)
       IF r2.status == "FAIL" THEN RAISE ConstraintException(r2.message) END IF
       seat = r2.data

       // Temporarily hold the seat before payment to prevent race conditions
       holdSeat(seat, request.passenger, HOLD_DURATION)

       r3 = confirmPayment(request.paymentInfo, seat.price)
       IF r3.status == "FAIL" THEN
          releaseSeatHold(seat)
          RAISE ConstraintException(r3.message)
       END IF

       allocateSeat(seat, request.passenger)
       bookingId = generateBookingId()
       logTestResult(bookingId, "CONFIRMED", currentTimestamp())
       RETURN CONFIRMED(bookingId)

    CATCH ConstraintException AS e
       logTestResult(request, "REJECTED - " + e.message, currentTimestamp())
       RETURN REJECTED(e.message)
    END TRY
 END FUNCTION

 // ---- Constraint Test-Case Matrix ----
 FUNCTION runConstraintTests():
    scenarios = [
       ("Valid all constraints", validPassenger, availableSeat, validPayment,
EXPECT="CONFIRMED"),
       ("Invalid passenger", badPassenger, availableSeat, validPayment,
EXPECT="REJECTED:C1"),
```

```
        ("No seats left", validPassenger, fullFlight, validPayment,
EXPECT="REJECTED:C2"),
        ("Payment declined", validPassenger, availableSeat, badPayment,
EXPECT="REJECTED:C3")
    ]
    FOR EACH s IN scenarios DO
        actual = processBooking(buildRequest(s))
        compareAndLog(s.EXPECT, actual)
    END FOR
 END FUNCTION

END Constraint_Based_Test_AirlineBooking
```

2. Process Flow Diagram

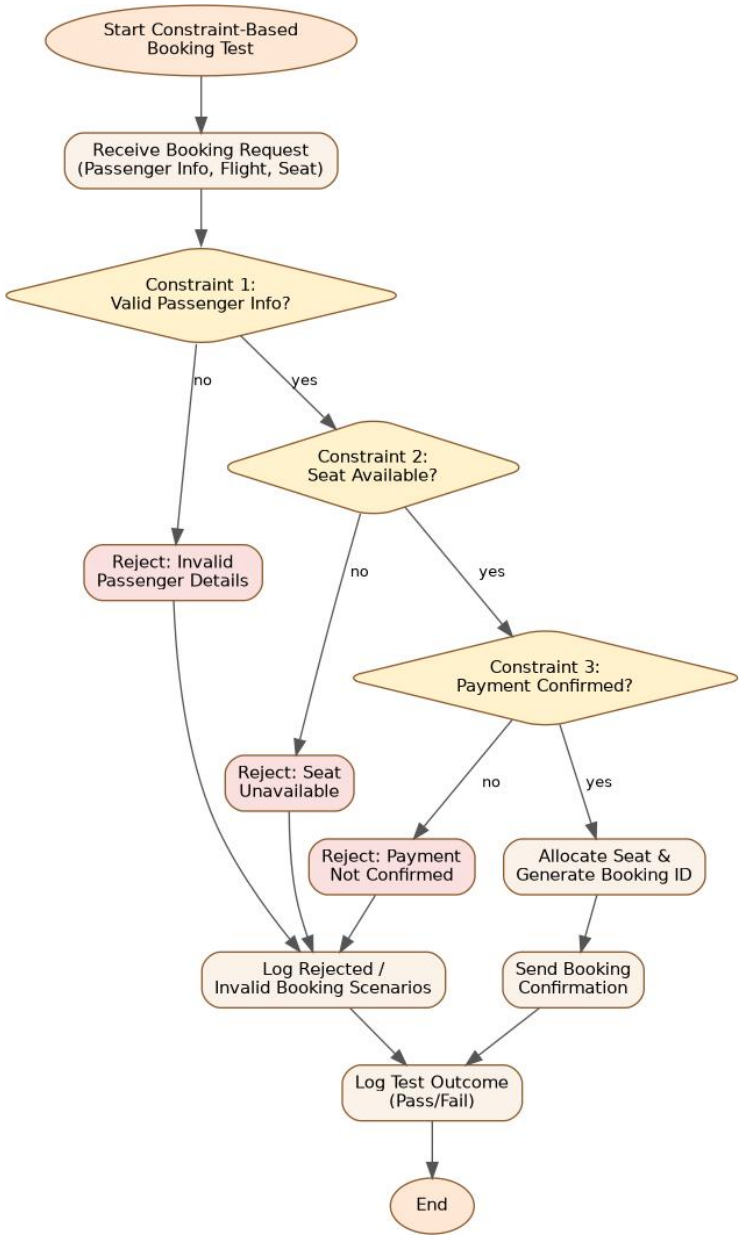

Figure 4: Process flow for Airline Reservation System testing procedure

3. Error Identification, Root Cause Analysis & Correction

Error Identified	Root Cause Analysis	Correction & Justification
Seat hold is placed before payment confirmation, but there is no timeout-expiry test to verify the hold is released if payment stalls indefinitely	holdSeat introduces a HOLD_DURATION constraint that was never exercised by a dedicated test scenario	Add a constraint test case that simulates a payment delay beyond HOLD_DURATION and asserts the seat is auto-released, preventing permanently locked inventory
Age validation (0-120) is a weak proxy for real passport/date-of-birth validation and can silently pass malformed data	The check substitutes a plausibility range for genuine cross-validation against passportId and date-of-birth format	Cross-validate age against passenger.dateOfBirth using a proper date parser rather than an isolated numeric range, closing the gap between the two data points
Constraint violation messages (C1/C2/C3) are logged only as free text, making automated regression comparison brittle	Root cause is embedding the constraint identifier inside a human-readable string instead of a structured error code	Return a structured {constraintId, message} object from each validator so downstream comparison (compareAndLog) is exact-match reliable, not string-matching dependent

4. Application of OOAD / QA Concepts

- Constraint-based testing explicitly enumerates each business rule (C1, C2, C3) and derives dedicated test scenarios, ensuring traceability from requirement to test case.
- Seat holding demonstrates concurrency-aware design — a key OOAD consideration to prevent race conditions when multiple passengers compete for the same seat.
- Each constraint maps to its own exception and validator function, following the single-responsibility principle.
- The expected-vs-actual scenario matrix operationalizes systematic, repeatable regression testing for booking logic.

Q5. Smart Library Management System

Modules / Features: Book Search, Book Reservation, Borrow/Return Services

Write pseudocode for integrating quality assurance and usability testing where the system evaluates both functional correctness (reservation and borrowing operations) and user experience factors such as response time, ease of navigation, and accessibility.

1. Pseudocode

```
BEGIN Integrated_QA_Usability_Test_Library

 // ---- FUNCTIONAL TEST: Book Search ----
 FUNCTION testBookSearch(query):
    t1 = currentTimestamp()
    results = BookSearch.execute(query)
    t2 = currentTimestamp()
    responseTime = t2 - t1
    correctness = validateSearchResults(results, query)
    IF NOT correctness THEN
       logDefect("BookSearch", "Incorrect or irrelevant results for: " + query)
    END IF
    RETURN {results, responseTime, correctness}
 END FUNCTION

 // ---- FUNCTIONAL TEST: Book Reservation (no double-booking) ----
 FUNCTION testReservation(userId, bookId):
    TRY
       book = DB.getBook(bookId)
       IF book IS NULL THEN
          RAISE ValidationException("Book not found")
       END IF
       IF book.status == "RESERVED" AND book.reservedBy != userId THEN
          RAISE ConflictException("Book already reserved by another user")
       END IF
       reserveBook(book, userId)
       logTestResult("Reservation", bookId, "PASS", currentTimestamp())
       RETURN "RESERVED"
    CATCH ValidationException, ConflictException AS e
       logDefect("Reservation", e.message)
       RETURN "FAILED"
    END TRY
 END FUNCTION

 // ---- FUNCTIONAL TEST: Borrow / Return ----
 FUNCTION testBorrowReturn(userId, bookId, action):
    TRY
       book = DB.getBook(bookId)
```

```
        IF action == "BORROW" THEN
            IF book.copiesAvailable <= 0 THEN
                RAISE ValidationException("No copies available")
            END IF
            decrementCopies(book)
            setDueDate(userId, book, DEFAULT_LOAN_PERIOD)
        ELSE IF action == "RETURN" THEN
            fine = calculateFine(userId, book)
            incrementCopies(book)
            closeLoanRecord(userId, book, fine)
        END IF
        logTestResult("BorrowReturn", bookId + ":" + action, "PASS", currentTimestamp())
    CATCH ValidationException AS e
        logDefect("BorrowReturn", e.message)
    END TRY
END FUNCTION

// ---- USABILITY TEST: Response Time, Navigation, Accessibility ----
FUNCTION testUsability(session):
    metrics = {}
    metrics.avgResponseTime = average(session.actionResponseTimes)
    metrics.clicksToReserve = session.countClicksFor("RESERVE_FLOW")
    metrics.accessibilityScore = runAccessibilityAudit(session.pageDOM)
    // e.g. screen-reader labels, contrast ratio, keyboard navigation
    RETURN metrics
END FUNCTION

// ---- INTEGRATION: Combine Functional + Usability Results ----
FUNCTION runIntegratedTestSuite(userId, bookId, query, session):
    searchResult = testBookSearch(query)
    reservationResult = testReservation(userId, bookId)
    borrowResult = testBorrowReturn(userId, bookId, "BORROW")
    usabilityMetrics = testUsability(session)

    report =
        { functional:
        {
            search: searchResult.correctness,
            reservation: reservationResult,
            borrow: borrowResult
        },
        usability: usabilityMetrics
    }

    IF usabilityMetrics.avgResponseTime > RESPONSE_TIME_THRESHOLD OR
       usabilityMetrics.accessibilityScore < ACCESSIBILITY_MIN_SCORE THEN
       report.usabilityVerdict = "NEEDS_IMPROVEMENT"
```

```
    ELSE
        report.usabilityVerdict = "ACCEPTABLE"
    END IF

    generateConsolidatedReport(report)
    RETURN report
 END FUNCTION

END Integrated_QA_Usability_Test_Library
```

2. Process Flow Diagram

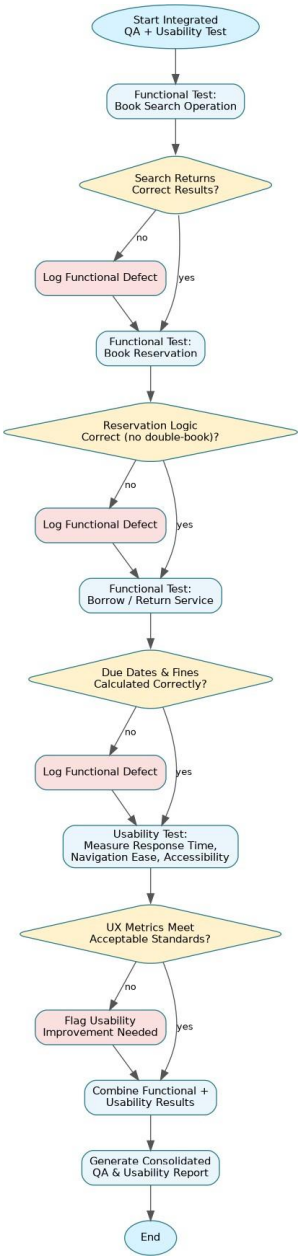

Figure 5: Process flow for Smart Library Management System testing procedure

3. Error Identification, Root Cause Analysis & Correction

Error Identified	Root Cause Analysis	Correction & Justification
Reservation conflict check only compares reservedBy to the current user but ignores books that are already fully on loan (copiesAvailable == 0) with no copies left to reserve against	testReservation was designed around the RESERVED status flag only, not the broader availability state shared with the borrow module	Check book.copiesAvailable and reservation queue state together before confirming a reservation, so functional correctness covers both reservation and inventory logic consistently
Accessibility is reduced to a single numeric accessibilityScore, which can mask specific failing criteria (e.g., missing alt-text vs. poor contrast)	runAccessibilityAudit was treated as a black-box score rather than itemized against WCAG success criteria	Return a breakdown of individual accessibility checks alongside the aggregate score, justified because a single number hides which usability defect needs fixing
Fine calculation on RETURN is invoked without first validating that a loan record actually exists for that user/book pair	closeLoanRecord assumes testBorrowReturn is always called in the correct BORROW-then-RETURN sequence	Add an explicit loan-existence check before calculateFine/closeLoanRecord and raise a ValidationException otherwise, preventing crashes from out-of-sequence test calls

4. Application of OOAD / QA Concepts

- Demonstrates integrated QA: functional correctness (search, reservation, borrow/return) and non-functional usability (response time, accessibility) are tested within a single consolidated suite, reflecting real-world quality assurance practice.
- Accessibility auditing operationalizes WCAG-style usability criteria into an automatable check, extending usability testing beyond speed/navigation metrics.
- State-conflict handling (double reservation, no available copies) reflects careful object state modeling in OOAD.
- The consolidated report pattern supports holistic decision-making, combining defect counts with UX verdicts for release readiness.